
QuestCash

Seguridad de la API: autenticación por token (JWT)

Entregable ED — Semana 2

*Prueba de que un endpoint protegido rechaza peticiones sin token
y las acepta con un token válido*

Proyecto	QuestCash
Componentes	questcash_web (API Flask) · questcash_mobile (Expo / React Native)
Módulo evaluado	auth_jwt.py — middleware @jwt_required
API	/api/v1
Algoritmo	JWT firmado con HS256
Autor	Armando Yael Contreras Bejarano
Fecha	17 de agosto de 2026

Documento generado a partir de la ejecución real de la batería de pruebas
descrita en la sección 3. Todas las respuestas HTTP mostradas son salidas literales de `curl`.

1. Resumen del entregable

El objetivo de la semana era proteger la API de QuestCash con autenticación basada en tokens. Al momento de esta entrega la implementación **ya estaba construida** en el backend (`questcash_web`) y consumida por la app móvil (`questcash_mobile`), por lo que el trabajo de esta semana consistió en **verificarla y documentar la evidencia** exigida por el entregable ED.

Requisito de la semana	Estado	Dónde está
Login que devuelva un JWT	✓ PASA	POST <code>/api/v1/auth/login</code> en <code>api.py</code>
Middleware que valide el token en cada petición	✓ PASA	decorador <code>@jwt_required</code> en <code>auth_jwt.py</code>
Endpoints sensibles protegidos	✓ PASA	20 rutas de <code>/api/v1</code> decoradas
Manejo de token expirado	✓ PASA	401 <code>token_expired</code>
Manejo de token inválido	✓ PASA	401 <code>invalid_token</code> / <code>missing_token</code>
Prueba: rechaza sin token / acepta con token	✓ PASA	sección 3, pruebas P1 y P2
Conexión asegurada con HTTPS	parcial	ver sección 4

Tabla 1: Cumplimiento de los objetivos de la semana 2.

Resultado global: las 10 pruebas ejecutadas pasaron. Un endpoint protegido devuelve **401 Unauthorized** sin token, con token manipulado, con token expirado y con token firmado por un tercero; y devuelve **200 OK** con los datos del usuario únicamente cuando el token es válido y está vigente.

2. Cómo está implementado

2.1 Emisión del token (login)

El endpoint público POST `/api/v1/auth/login` valida las credenciales contra el hash almacenado (`werkzeug.security.check_password_hash`) y, si son correctas, emite un JWT firmado con HS256:

Listing 1: `auth_jwt.py` — generación del token.

```

1 def generate_token(usuario_id):
2     now = datetime.now(timezone.utc)
3     payload = {
4         "sub": str(usuario_id),
5         "iat": now,
6         "exp": now + timedelta(days=current_app.config["JWT_EXP_DAYS"]),
7     }
8     return jwt.encode(payload, current_app.config["JWT_SECRET_KEY"], algorithm="HS256")

```

El login además comparte el mismo contador de intentos fallidos que el login web (`MAX_LOGIN_INTENTOS` / `BLOQUEO_MINUTOS`), de modo que la API no abre un segundo vector de fuerza bruta contra la misma cuenta: tras N intentos responde **429** con `{"error": "locked"}`.

2.2 Middleware de validación

Toda ruta sensible se decora con `@jwt_required`. El decorador extrae el token del encabezado `Authorization: Bearer <token>`, lo verifica y carga el usuario en `g.api_usuario` antes de ceder el control a la vista. Cada caso de fallo tiene su propio código de error:

Listing 2: `auth_jwt.py` — middleware de autenticación.

```

1 def jwt_required(vista):
2     """Exige un Bearer token valido y carga g.api_usuario."""
3     @wraps(vista)
4     def wrapped(*args, **kwargs):
5         token = _extraer_token()
6         if not token:
7             return jsonify({"error": "missing_token"}), 401
8
9         try:
10            payload = decode_token(token)
11        except jwt.ExpiredSignatureError:
12            return jsonify({"error": "token_expired"}), 401
13        except jwt.InvalidTokenError:
14            return jsonify({"error": "invalid_token"}), 401
15
16        usuario = Usuario.query.get(int(payload["sub"]))
17        if usuario is None:
18            return jsonify({"error": "invalid_token"}), 401
19
20        g.api_usuario = usuario
21        return vista(*args, **kwargs)
22
23    return wrapped

```

Nótese que la validación de firma y de expiración la hace PyJWT dentro de `jwt.decode()`, no código propio: es la biblioteca la que recalcula el HMAC-SHA256 y compara el claim `exp` contra la hora actual. El decorador solo traduce esas excepciones a respuestas JSON. Además se verifica que el usuario del claim `sub` siga existiendo en la base de datos, de modo que un token de una cuenta eliminada tampoco sirve.

2.3 Superficie protegida

De los 23 endpoints de `/api/v1`, **20 exigen token**. Los únicos tres públicos son los que por definición no pueden exigirlo:

Endpoint	Método	Protección
/auth/register	POST	pública (crea la cuenta y emite token)
/auth/login	POST	pública (valida credenciales, emite token)
/auth/me	GET	@jwt_required
/dashboard	GET	@jwt_required
/quests	GET, POST	@jwt_required
/quests/<id>	GET, PUT, DELETE	@jwt_required
/quests/<id>/cancel	POST	@jwt_required
/quests/<id>/movimientos	GET, POST	@jwt_required
/quests/<id>/colaboradores	GET, POST	@jwt_required
/gastos	GET, POST	@jwt_required
/categorias-gasto	GET	@jwt_required
/insignias	GET	@jwt_required
/notificaciones	GET	@jwt_required
/notificaciones/<id>/leer	POST	@jwt_required
/perfil	GET, PATCH	@jwt_required

Tabla 2: Endpoints de la API y su nivel de protección.

Sobre el token, los endpoints que tocan recursos de un usuario concreto aplican además una verificación de *pertenencia*: por ejemplo GET /quests/<id> responde **403 forbidden** si el usuario autenticado no participa en esa meta. Autenticar no es lo mismo que autorizar, y ambas capas están presentes.

2.4 Consumo desde la app móvil

En `questcash_mobile` el token se guarda en `expo-secure-store` (Keychain en iOS, Keystore en Android; no en `AsyncStorage`, que es texto plano) y un interceptor de Axios lo adjunta automáticamente a cada petición:

Listing 3: `src/services/api.ts` — inyección del token y reacción al 401.

```

1 client.interceptors.request.use(async (config) => {
2   const token = await SecureStore.getItemAsync(TOKEN_KEY);
3   if (token) {
4     config.headers = config.headers ?? {};
5     config.headers.Authorization = `Bearer ${token}`;
6   }
7   return config;
8 });
9
10 client.interceptors.response.use(
11   (response) => response,
12   (error) => {
13     if (error.response?.status === 401) {
14       onUnauthorized?(); // -> AuthContext borra el token y regresa al login
15     }
16     return Promise.reject(error);
17   }
18 );

```

Con esto el manejo de expiración queda cerrado de extremo a extremo: cuando el token vence, el backend responde **401 token_expired**, el interceptor lo detecta, `AuthContext` borra el token del

almacenamiento seguro y el navegador raíz devuelve al usuario a la pantalla de *login*. No hay estado colgado ni pantallas mostrando datos de una sesión ya inválida.

3. Evidencia de pruebas

3.1 Cómo se ejecutaron

Se levantó una instancia de Flask que monta el decorador `@jwt_required` y la función `generate_token()` tal cual están en el proyecto (archivos `auth_jwt.py` y `config.py` sin modificar), sobre dos endpoints representativos: `/api/v1/auth/me` y `/api/v1/dashboard`. Contra esa instancia se corrió la batería `pruebas.sh` con `curl`. Las salidas siguientes son literales.

Nota de reproducibilidad. El servidor de prueba sustituye únicamente la capa de acceso a datos por un objeto `Usuario` en memoria; la lógica de autenticación evaluada es la del proyecto, sin cambios. El script `pruebas.sh` apunta a `http://127.0.0.1:5001/api/v1`, que es exactamente la URL base que usa la app móvil en desarrollo, así que puede ejecutarse igual contra el backend completo levantando `app.py` en el puerto 5001.

3.2 Resumen de resultados

#	Escenario	Esperado	Obtenido
P0	Login con credenciales válidas	200 + token	200 ✓ PASA
P0b	Login con contraseña incorrecta	401	401 ✓ PASA
P1	Endpoint protegido SIN token	401	401 ✓ PASA
P2	Endpoint protegido CON token válido	200	200 ✓ PASA
P3	Token manipulado (sub alterado)	401	401 ✓ PASA
P4	Token expirado	401	401 ✓ PASA
P5	Token firmado con otra clave secreta	401	401 ✓ PASA
P6	Esquema Basic en vez de Bearer	401	401 ✓ PASA
P7	Segundo endpoint protegido (/dashboard)	401 / 200	401 / 200 ✓ PASA
P8	Estructura del token emitido	HS256, sub/iat/exp	✓ PASA

Tabla 3: 10 de 10 pruebas superadas.

3.3 P0 — El login devuelve un JWT

```
$ curl -i -X POST /api/v1/auth/login -d '{"correo":"...", "password":"..."}'

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 202

{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwiaWF0IjoxNzgzOTI2MzUxLCJleHAiOiJlMTgzNTF9.NyriFCFB8aGz-iEx3RF6r7Zo9dh0MN3vvnSEhoFh91s",
"user":{"correo":"armando@questcash.mx","id":1}}
```

Con credenciales incorrectas (P0b) el mismo endpoint responde:

```
$ curl -i -X POST /api/v1/auth/login -d '{"correo":"...", "password":"incorrecta"}'

HTTP/1.1 401 UNAUTHORIZED
Content-Type: application/json

{"error":"invalid_credentials"}
```

3.4 P1 — Sin token: la API rechaza (prueba central del entregable)

```
$ curl -i http://127.0.0.1:5001/api/v1/auth/me

HTTP/1.1 401 UNAUTHORIZED
Content-Type: application/json
Content-Length: 26

{"error":"missing_token"}
```

Sin encabezado `Authorization` la petición ni siquiera llega al cuerpo de la vista: el decorador corta antes y no se filtra ningún dato del usuario.

3.5 P2 — Con token válido: la API acepta (prueba central del entregable)

```
$ curl -i http://127.0.0.1:5001/api/v1/auth/me -H "Authorization: Bearer $TOKEN"

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 74

{"user":{"correo":"armando@questcash.mx", "id":1, "nombre":"Armando Yael"}}
```

Misma URL, mismo método, mismo servidor: la única diferencia entre P1 y P2 es el encabezado `Authorization`. Eso es exactamente lo que pide el entregable ED.

3.6 P3 — Token manipulado

Se tomó un token legítimo y se reescribió su *payload* cambiando `"sub":"1"` por `"sub":"2"` (intento de suplantar a otro usuario), conservando la firma original:

```
$ curl -i /api/v1/auth/me -H "Authorization: Bearer <payload alterado>"

HTTP/1.1 401 UNAUTHORIZED
Content-Type: application/json

{"error":"invalid_token"}
```

La firma HMAC deja de coincidir con el contenido y PyJWT lanza `InvalidTokenError`. Un JWT es legible por cualquiera, pero no modificable sin la clave secreta: esta prueba lo demuestra.

3.7 P4 — Token expirado

Token emitido con `exp` 10 segundos en el pasado:

```
$ curl -i /api/v1/auth/me -H "Authorization: Bearer <token vencido>"

HTTP/1.1 401 UNAUTHORIZED
Content-Type: application/json

{"error": "token_expired"}
```

El código de error se distingue de `invalid_token` a propósito: le permite al cliente saber que las credenciales eran correctas pero la sesión caducó, y actuar en consecuencia (renovar o mandar al login) en vez de mostrar un error genérico.

3.8 P5 — Token firmado con otra clave

Simula a un atacante que construye un JWT bien formado con su propia clave secreta:

```
$ curl -i /api/v1/auth/me -H "Authorization: Bearer <firmado por un tercero>"

HTTP/1.1 401 UNAUTHORIZED
Content-Type: application/json

{"error": "invalid_token"}
```

3.9 P6 — Esquema de autenticación incorrecto

```
$ curl -i /api/v1/auth/me -H "Authorization: Basic YWJjOjEyMw=="

HTTP/1.1 401 UNAUTHORIZED
Content-Type: application/json

{"error": "missing_token"}
```

El middleware exige explícitamente el prefijo `Bearer` ; cualquier otro esquema se trata como ausencia de token.

3.10 P7 — La protección no es un caso aislado

Se repitió el par sin token / con token contra un segundo endpoint protegido:

```
$ curl -o /dev/null -w "%{http_code}" /api/v1/dashboard

sin token    -> 401
con Bearer   -> 200
```

3.11 P8 — Estructura del token emitido

```
Decodificación del token (base64url, sin verificar firma)

header : {"alg": "HS256", "typ": "JWT"}
payload: {"sub": "1", "iat": 1786926351, "exp": 1789518351}
iat     : 2026-08-17T00:25:51+00:00
exp     : 2026-09-16T00:25:51+00:00
```

```

vigencia: 30.0 días
firma    : NyrifCFB8aGz-iEx3RF6... (43 chars)

```

El token contiene solo el identificador del usuario y las marcas de tiempo: no lleva contraseña, ni correo, ni datos personales. Como el payload de un JWT viaja en base64url (legible, no cifrado), incluir ahí información sensible sería un error; aquí no se incluye.

4. Estado de la conexión segura (HTTPS)

Este es el punto del objetivo semanal que **aún no está cerrado** y conviene declararlo con claridad en vez de darlo por hecho.

Entorno	Protocolo	Situación
Desarrollo (Expo + Flask)	http://	SERVER_ORIGIN se arma como http://<host>:5001. Tráfico en la red local.
Docker (docker-compose)	http://	El servicio web expone el puerto 80 sin TLS.
Cookies de sesión web	—	SESSION_COOKIE_SECURE = False en config.py, con el comentario “ <i>cambia a True si usas HTTPS</i> ”.

Tabla 4: Estado actual del transporte.

En desarrollo local esto es aceptable y es la práctica habitual: el certificado autofirmado rompe el cliente de Expo y no aporta seguridad real en localhost. Pero el token JWT viaja en un encabezado en texto plano, así que **sobre HTTP cualquiera en la misma red puede interceptarlo y reutilizarlo**. La autenticación por token y el TLS no son alternativas: el token identifica *quién* pide, y el TLS impide que alguien más lo copie en el camino.

Pasos pendientes para cerrar el punto

1. Poner un proxy inverso (Nginx o Caddy) delante de Gunicorn y terminar TLS ahí, con certificado de Let’s Encrypt. Caddy lo hace automáticamente con dos líneas de configuración.
2. Redirigir todo http:// a https:// y añadir la cabecera Strict-Transport-Security (HSTS).
3. Cambiar SESSION_COOKIE_SECURE a True en producción.
4. En la app móvil, dejar que App Transport Security (iOS) y cleartextTrafficPermitted=false (Android) bloqueen HTTP en el build de producción, permitiéndolo solo en desarrollo.
5. Mover JWT_SECRET_KEY a variable de entorno con un valor fuerte. Hoy cae por defecto a SECRET_KEY, que a su vez cae a "dev_key" si no está definida; en producción esa cadena permitiría a cualquiera firmar tokens válidos.

5. Conclusiones

1. **El entregable ED está cubierto.** Las pruebas P1 y P2 muestran, sobre la misma URL y con la única diferencia del encabezado `Authorization`, que un endpoint protegido responde **401** sin token y **200** con token válido.
2. **La cobertura es amplia, no simbólica.** 20 de 23 endpoints exigen token; los 3 públicos son los de registro y login. Encima de la autenticación hay verificación de pertenencia (**403**) en los recursos por usuario.
3. **Los tres modos de fallo están diferenciados** (`missing_token`, `invalid_token`, `token_expired`), lo que le permite al cliente móvil reaccionar distinto a cada uno.
4. **El ciclo de expiración está cerrado de extremo a extremo:** el backend devuelve 401, el interceptor de Axios lo detecta, se borra el token del almacenamiento seguro y la app vuelve al login.
5. **Queda pendiente el HTTPS** y el endurecimiento de la clave secreta. Es el único punto del objetivo semanal que no está terminado y está detallado en la sección 4.

Sugerencia para la siguiente iteración: la vigencia de 30 días es cómoda para el usuario pero larga para un token que no se puede revocar. El patrón habitual es un *access token* corto (15–60 minutos) más un *refresh token* de larga duración almacenado en la base de datos, que sí puede invalidarse al cerrar sesión o ante un incidente.

Anexo A — Salida completa de la batería de pruebas

```
$ ./pruebas.sh

=====
P0 - LOGIN con credenciales validas => devuelve JWT (200)
=====
HTTP/1.1 200 OK
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","user":{"correo":"armando@questcash.mx","id":1}}

=====
P0b - LOGIN con contrasena incorrecta => 401 invalid_credentials
=====
HTTP/1.1 401 UNAUTHORIZED
{"error":"invalid_credentials"}

=====
P1 - GET /auth/me SIN token => RECHAZA (401 missing_token)
=====
HTTP/1.1 401 UNAUTHORIZED
{"error":"missing_token"}

=====
P2 - GET /auth/me CON token valido => ACEPTA (200)
=====
HTTP/1.1 200 OK
{"user":{"correo":"armando@questcash.mx","id":1,"nombre":"Armando Yael"}}

=====
P3 - Token MANIPULADO (payload sub 1 -> 2, firma original) => 401 invalid_token
=====
HTTP/1.1 401 UNAUTHORIZED
{"error":"invalid_token"}

=====
P4 - Token EXPIRADO => 401 token_expired
=====
HTTP/1.1 401 UNAUTHORIZED
{"error":"token_expired"}

=====
P5 - Token firmado con OTRA clave secreta => 401 invalid_token
=====
HTTP/1.1 401 UNAUTHORIZED
{"error":"invalid_token"}

=====
P6 - Esquema de cabecera incorrecto (Basic en vez de Bearer) => 401 missing_token
=====
HTTP/1.1 401 UNAUTHORIZED
{"error":"missing_token"}

=====
P7 - Segundo endpoint protegido /dashboard: sin token vs con token
=====
sin token -> 401
con Bearer -> 200

=====
P8 - Contenido del token emitido (header y payload decodificados)
=====
header : {"alg": "HS256", "typ": "JWT"}
payload : {"sub": "1", "iat": 1786926351, "exp": 1789518351}
iat      : 2026-08-17T00:25:51+00:00
exp      : 2026-09-16T00:25:51+00:00
```

```
vigencia: 30.0 dias  
firma    : NyrifCFB8aGz-iEx3RF6... (43 chars)
```

Archivos de referencia del proyecto

- `questcash_web/auth_jwt.py` — generación y validación del token, decorador `@jwt_required`.
- `questcash_web/api.py` — blueprint `/api/v1`, login y endpoints protegidos.
- `questcash_web/config.py` — `JWT_SECRET_KEY`, `JWT_EXP_DAYS`.
- `questcash_mobile/src/services/api.ts` — interceptores de Axios.
- `questcash_mobile/src/context/AuthContext.tsx` — sesión, logout automático ante 401.